

Agentic AI Development Stack

Quick-reference cheat sheet for your Codespace environment: Docker, Node, Python, Claude Code CLI, and the core Python agentic AI packages

1. Core Environment

Tool	What it is	Role in your agentic AI workflow
Node.js (v20)	JavaScript runtime	Required to run the Claude Code CLI (built on Node) and any JavaScript-based MCP servers you connect to.
Python (3.11)	Programming language runtime	The language almost all agentic AI frameworks (LangChain, LangGraph, CrewAI, ADK) are written in and used through.
Docker (Docker-in-Docker)	Containerization engine	Lets you build and run isolated containers inside your Codespace - e.g. testing an MCP server the same way Stacklok's ToolHive isolates them, or containerizing an agent for deployment.
Claude Code CLI	Agentic coding assistant	Runs directly in your terminal, reads/edits your repo files, executes commands, and can be configured to route through OpenRouter or other gateways.

2. LLM Provider SDKs

These are the foundational libraries that let your Python code actually talk to a model. Every framework below (LangChain, CrewAI, etc.) is built on top of these.

Package	Purpose	Simple Example
anthropic	Official SDK to call Claude models directly via the Anthropic API.	<pre>client = anthropic.Anthropic() client.messages.create(model='claude-sonnet-4-6', ...)</pre>
openai	Official SDK to call OpenAI models (GPT-4o, etc.) - also used by many tools as a generic "OpenAI-compatible" client for other providers.	<pre>client = OpenAI() client.chat.completions.create(...)</pre>
google-genai	Official SDK to call Google's Gemini models directly. The lightweight, single-call counterpart to google-adk (which is a full agent orchestration framework, not just a model client).	<pre>client = genai.Client(api_key=...) client.models.generate_content(model='gemini-2.5-flash', ...)</pre>

3. Agent Orchestration Frameworks

These packages give you the building blocks to chain prompts, manage multi-step agent reasoning, add memory, and coordinate multiple agents together - going beyond a single API call.

LangChain / LangGraph family

Package	Purpose
langchain	The original framework for chaining LLM calls, tools, and memory into structured pipelines.
langchain-core	Shared base abstractions (prompts, output parsers, runnables) that other LangChain packages build on.
langchain-anthropic	The Claude-specific connector so LangChain pipelines can call Anthropic models.
langchain-openai	The OpenAI-specific connector, same idea, for GPT models.
langchain-protocol	Shared protocol/interface definitions used across LangChain packages for interoperability.
langgraph	Builds agent workflows as a graph of steps/nodes - the modern recommended way to build stateful, multi-step agent loops (e.g. "think, act, observe, repeat").
langgraph-checkpoint	Saves and restores agent state at each step, so a long-running agent can pause/resume or recover from failure.
langgraph-prebuilt	Ready-made common agent patterns (like a ReAct agent) so you don't build the graph from scratch every time.
langgraph-sdk	Client SDK for interacting with a deployed LangGraph server/service remotely.

Example: minimal LangGraph agent loop

```
from langgraph.prebuilt import create_react_agent
from langchain_anthropic import ChatAnthropic
agent = create_react_agent(ChatAnthropic(model='claude-sonnet-4-6'), tools=[my_tool])
agent.invoke({'messages': [('user', 'What is the weather in Austin?')]})
```

CrewAI family

Package	Purpose
crewai	A framework focused on multi-agent "crews" - you define several agents with distinct roles (e.g. Researcher, Writer, Reviewer) that collaborate on a task.
crewai-core	The core engine/runtime that crewai depends on.
crewai-cli	Command-line tool to scaffold, run, and manage CrewAI projects from the terminal.

Google ADK

Package	Purpose
google-adk	Google's code-first agent framework, optimized for Gemini/Google Cloud but model-agnostic. Supports the A2A (Agent-to-Agent) protocol for agents to communicate with each other. Good for comparing against LangGraph/CrewAI design patterns.

4. Model Context Protocol (MCP)

Package	Purpose
mcp	The official Python SDK for building or connecting to MCP servers - the standardized way an agent connects to external tools and data sources (databases, APIs, file systems), instead of writing one-off custom integrations for every AI client.

Example: minimal MCP server exposing one tool

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP('demo-server')

@mcp.tool()
def get_dti(income: float, debt: float) -> float:
    return round((debt / income) * 100, 2)

mcp.run()
```

5. RAG / Retrieval Stack

These packages let an agent look up and reason over your own documents/data instead of relying only on the model's built-in knowledge - the practical implementation of what we discussed as the semantic layer + retrieval pattern.

Package	Purpose
llama-index	A framework purpose-built for RAG - connecting LLMs to your own documents, databases, or APIs for retrieval-augmented answers.
llama-index-core	The core engine that all llama-index integration packages plug into.
llama-index-embeddings-openai	Uses OpenAI's embedding models to convert text into vectors for semantic search.
llama-index-llms-openai	Connector so llama-index can use OpenAI models as the reasoning/generation layer.
llama-index-workflows	Lets you define multi-step RAG or agent workflows as an explicit, debuggable sequence of steps.
llama-index-instrumentation	Observability/tracing hooks so you can see what a llama-index pipeline actually did, step by step.
chromadb	An open-source vector database - stores embeddings and lets you search for the most semantically similar chunks of text quickly (the retrieval half of RAG).

Example: minimal RAG lookup with ChromaDB

```
import chromadb
client = chromadb.Client()
collection = client.create_collection('docs')
collection.add(documents=['Bank policy: DTI over 43% is high risk'], ids=['doc1'])
results = collection.query(query_texts=['What DTI counts as high risk?'], n_results=1)
```